Infix, Prefix and Postfix

Expressions and Stack | Easy explanation with examples

Infix

```
A + B
```

Prefix

```
+ A B
```

Postfix

```
A B +
```

What is an Expression?

- An expression is a small maths sentence, like A + B * C.
- Operand = the value or letter (A, B, 5).
- Operator = the action sign (+, -, *, /).



Three Ways to Write an Expression

1

Infix

The operator is in the middle. This is how we write maths every day.

```
A + B
```

2

Prefix

The operator comes before the operands. (Pre = before)

```
+ A B
```

3

Postfix

The operator comes after the operands. (Post = after)

```
A B +
```

All three mean the same thing. Only the place of the operator changes.

Why Do We Need Prefix and Postfix?

Infix

- Needs brackets
- Needs precedence rules (* before +)
- Hard for a computer to read

Prefix and Postfix

- No brackets needed
- No precedence rules needed
- Easy for a computer, with a stack

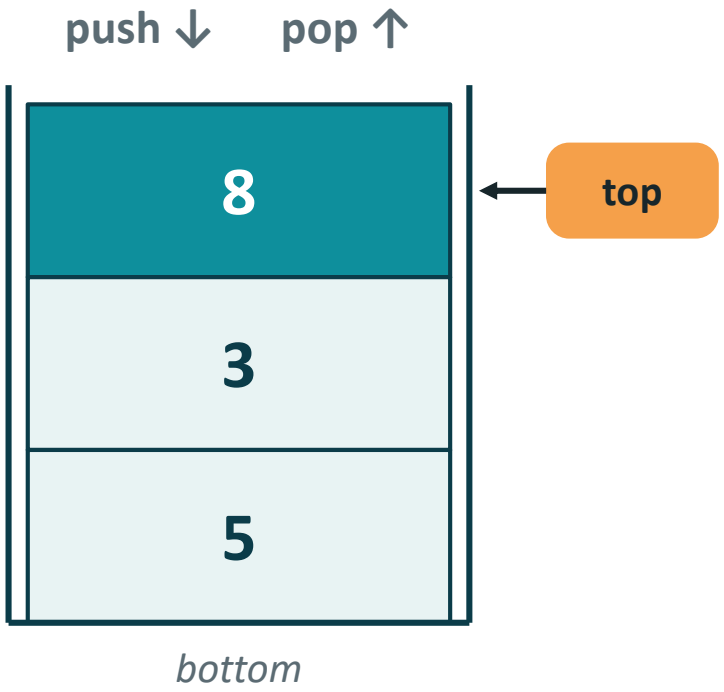
Operator Priority (Precedence)

Priority	Operators	Meaning
Highest	( )	Do the brackets first
High	* /	Multiply and divide next
Low	+ −	Add and subtract last

Example
2 + 3 * 4 = 14, not 20. Because * is done first.

Same priority?
Go from left to right. 8 - 3 + 2 = 7.

What is a Stack?



- A stack is a pile. Think of plates on a table.
- LIFO: Last In, First Out.
- push = put an item on the top.
- pop = take the top item out.
- top = look at the top item.

Try it
push(9) puts 9 on top. pop() removes 9.

Infix to Postfix: The Rules

1. Read the expression from left to right.
2. Letter or number: send it to the output.
3. (: push it on the stack.
4.) : pop to the output until you see (. Then remove (.
5. Operator: pop operators of higher or same priority to the output. Then push it.
6. At the end: pop everything from the stack to the output.

Example 1: A + B * C

Symbol	Stack	Output	What we did
A	empty	A	Letter goes to output
+	+	A	Stack is empty, so push +
B	+	A B	Letter goes to output
*****	+ *	A B	* is higher than +, so just push
C	+ *	A B C	Letter goes to output
end	empty	A B C * +	Pop all (top first)

Postfix: A B C * +

Example 2: (A + B) * C

Symbol	Stack	Output	What we did
(	(		Push (
A	(	A	Letter goes to output
+	(+	A	Top is (, so push +
B	(+	A B	Letter goes to output
)	empty	A B +	Pop until (, then remove (
*****	*	A B +	Stack is empty, push *
C	*	A B + C	Letter goes to output
end	empty	A B + C *	Pop all

Postfix: A B + C *

Solve a Postfix: 2 3 4 * +

Rule

Number: push it. Operator: pop two numbers, calculate, push the answer.

Symbol	Stack	What we did
2	2	Push 2
3	2 3	Push 3
4	2 3 4	Push 4
*****	2 12	Pop 4 and 3. 3 * 4 = 12. Push 12
+	14	Pop 12 and 2. 2 + 12 = 14. Push 14

Answer = 14

The first number popped is the RIGHT one. For 8 2 -, pop 2 then 8: 8 - 2 = 6.

Infix to Prefix: A + B * C

1. Reverse the expression

```
C * B + A
```

2. Convert it to postfix (same stack rules)

```
C B * A +
```

3. Reverse the result

```
+ A * B C
```

Prefix: + A * B C

With brackets: when you reverse, swap them too. (becomes) and) becomes (.

Solve a Prefix: + 2 * 3 4

Rule

Read from RIGHT to LEFT. Number: push. Operator: pop two, calculate, push.

Symbol	Stack	What we did
4	4	Push 4
3	4 3	Push 3
*****	12	Pop 3 and 4. 3 * 4 = 12. Push 12
2	12 2	Push 2
+	14	Pop 2 and 12. 2 + 12 = 14. Push 14

Answer = 14

Here the first number popped is the LEFT one. For - 8 2, pop 8 then 2: 8 - 2 = 6.

C++ Program - Part 1: Priority and main()

```
#include <iostream>
#include <stack>
using namespace std;

int prec(char c) {
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;   // '(' gets 0
}

string toPostfix(string s); // next slide

int main() {
    string exp = "(A+B)*C";
    cout << toPostfix(exp) << endl;
    return 0;
}
```

prec()

Gives a priority number. * and / = 2, + and - = 1, (= 0.

stack<char>

C++ has a ready-made stack. Add #include <stack>.

Output

```
AB+C*
```

C++ Program - Part 2: toPostfix()

```
string toPostfix(string s) {
    stack<char> st;
    string out = "";
    for (char c : s) {
        if (isalnum(c)) out += c;
        else if (c == '(') st.push(c);
        else if (c == ')') {
            while (st.top() != '(') {
                out += st.top(); st.pop();
            }
            st.pop();
        } else {
            while (!st.empty() && prec(st.top()) >= prec(c)) {
                out += st.top(); st.pop();
            }
            st.push(c);
        }
    }
    while (!st.empty()) { out += st.top(); st.pop(); }
    return out;
}
```

Letter
Add it to out.

(
Push it on the stack.

)
Pop to out until (.

Operator
Pop higher or same priority, then push.

C++ Program - Part 3: Solve a Postfix

```
int evalPostfix(string s) {
    stack<int> st;
    for (char c : s) {
        if (isdigit(c)) st.push(c - '0');
        else {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();
            if (c == '+') st.push(a + b);
            if (c == '-') st.push(a - b);
            if (c == '*') st.push(a * b);
            if (c == '/') st.push(a / b);
        }
    }
    return st.top();
}

// in main():
cout << evalPostfix("234*+");
```

c - '0'

Turns the character '4' into the number 4.

b first, then a

b is the right number, a is the left. So a - b is correct.

Output

```
14
```

Quick Comparison

Type	Operator is	Brackets	(A+B)*C becomes	Solve with stack
Infix	In the middle	Needed	`(A + B) * C`	Hard (needs rules)
Prefix	Before operands	Not needed	`* + A B C`	Right to left
Postfix	After operands	Not needed	`A B + C *`	Left to right

Tip: Computers love postfix. One scan and one stack is enough to solve it.

Remember

Infix, Prefix, Postfix

Operator in the middle, before, after.

Stack = LIFO

push and pop happen at the top.

Infix to Postfix

Letters go out. Operators wait on the stack.

Solve Postfix

Number: push. Operator: pop two, push result.